

Capture the Flag: A Real-Time Robotic Pursuit-Evasion Game Using Vision-Based Autonomous Agents

Arooba Farhan*, Shariq Kazmi*, Mahnoor Kashif* *Lahore University of Management Sciences, Lahore, Pakistan
 {26100128, 26100101, 26100362}@lums.edu.pk

Abstract

This project presents the design, simulation, and real-world implementation of a robotic Capture the Flag (CTF) game using the RoboMaster platform, aimed at integrating vision-based perception and autonomous decision-making. The game scenario involves two mobile agents—an attacker and a defender—operating on an 3x3 grid-based arena. The attacker’s objective is to acquire a centrally located flag and deliver it to a designated drop zone while evading capture. The defender, equipped with a real-sense camera, tracks the attacker in real-time and attempts interception through vision based red-object detection.

Key contributions of this work include the development of a robust HSV-based color segmentation pipeline for visual tracking, an A*-based dynamic path-planning algorithm enabling real-time rerouting under threat, and a multi-threaded control architecture that ensures seamless concurrent behavior of both agents. The simulation phase, implemented in Python, models strategic interactions with randomized obstacle placement and a human-controlled attacker. The hardware implementation translates these strategies into a physical arena using two RoboMaster robots equipped with on-board vision systems.

I. INTRODUCTION

Capture the Flag (CTF) games offer a dynamic framework to explore autonomy, perception, and strategy in mobile robots. In our version of CTF, two autonomous ground robots; the attacker and the defender—compete in a bounded 3x3 grid. The attacker must navigate to a central flag, pick it up, and carry it to a designated drop zone at the far end. Simultaneously, the defender attempts to block this operation by tracking and intercepting the attacker. This setup

explores the challenges of real-time pursuit-evasion in physical environments, contrasting human intuition and variability against deterministic autonomous planning and tracking algorithms. Our implementation integrates several robotics challenges into a single coordinated system.

Visual detection and tracking are performed using the RoboMaster SDK’s built-in camera with depth perception capabilities. We apply HSV-based color masking to detect the red flag and attacker robot, and use the SDK’s depth information to localize objects in 3D space with reasonable accuracy. This choice simplifies the hardware setup while still enabling effective perception in a controlled environment. For navigation, the defender employs an A* search algorithm to generate grid-based paths toward the flag and drop zone. To adapt to the dynamic presence of the attacker, we introduce a re-planning mechanism: the defender maps the shortest distance from the position of the attacker at every instance while the attacker is operated manually through a set of commands. This responsiveness prioritizes safety and adaptability. For navigation, the attacker employs an A* search algorithm to generate grid-based paths toward the flag and drop zone. To adapt to the dynamic presence of the defender, we introduce a re-planning mechanism: if the defender obstructs the current path, the attacker aborts it and computes an alternative route in real time. This responsiveness prioritizes safety and adaptability over strictly shortest-path behavior.

To support this behavior, both robots run asynchronous control threads. The attacker simultaneously handles visual processing, path planning, and motion control without blocking tasks. The defender continuously monitors the attacker’s position while executing its own motion strategy. This separation of control pipelines prevents perception delays from stalling movement decisions, which is critical in time-sensitive adversarial scenarios.

To support this behavior, the defender simultaneously handles visual processing, path planning, and motion control without blocking tasks. The defender continuously monitors the attacker’s position while executing its own motion strategy. This separation of control pipelines prevents perception delays from stalling movement decisions, which is critical in time-sensitive adversarial scenarios. Overall, our CTF setup serves as a compact, interactive environment for evaluating real-time perception, strategic movement, and multi-agent coordination in autonomous mobile robots. The system highlights trade-offs between reactive decision-making and planned behavior, and opens up possibilities for future work involving larger teams, more complex maps, or dynamic role switching.

II. RELATED WORK

Robotic perception and planning often begin with color-based detection, which provides a lightweight means of identifying objects of interest. Pour Yousefian Barfeh and Ramos [1] introduced a partition-based algorithm that processes live video frames from a simple onboard camera to detect a target color in real time. Their robot reacts immediately to the detected color region without costly image transformations, achieving rapid left/right motion decisions. Building on this, Bhat et al. [5] demonstrated a full processing pipeline, from webcam image acquisition through MATLAB color segmentation to microcontroller-driven actuation, to track colored objects over serial links. While both works confirm the viability of color masking for reactive behaviors, they rely on 2D pixel information alone. In contrast, our system leverages the RoboMaster SDK’s depth camera to obtain true 3D positions of the flag and opponent, enabling more precise localization and reducing false positives due to background clutter.

In the domain of human-following robots, Artono et al. [2] designed a trolley that follows a person by recognizing clothing color via OpenCV, translating image pixel offsets into PWM signals for Arduino-controlled motors. Their platform also integrates ultrasonic sensors to avoid collisions when the tracked person comes within 30 cm. This work motivated our safety-aware evasion: like their trolley, our defender uses depth data to maintain a safe intercept distance, but unlike a single-leader/follower scenario, it must predict and block the attacker’s path in a competitive setting rather than passively follow.

Path planning on RoboMaster platforms was explored by a TJPRC group [3], who evaluated multiple start-to-end trajectories to minimize cycle time for a six-axis arm. Their analysis highlights how software constraints and environment models influence optimal path selection. We adapt their evaluation mindset to a discrete 3×3 grid, employing an A* algorithm to compute optimal routes, but extend it with a dynamic re-planning mechanism that aborts and recomputes paths whenever the defender obstructs the attacker. This ensures our attacker remains both efficient and reactive under adversarial pressure.

Line-following robots, such as the one by Anilkumar et al. [4], separate sensing (IR and ultrasonic), decision (junction mapping), and actuation into distinct control loops, enabling reliable path tracing on marked lines. Inspired by this modularity, our multi-threaded control architecture runs perception, planning, and motion simultaneously on each RoboMaster, preventing delays in one component from stalling the others—critical in time-sensitive pursuit/evasion scenarios.

Collectively, these prior works establish the feasibility of color-based detection, single-agent following, cycle-time path optimization, and sensor-actuated control loops. Our contribution integrates and advances them into a unified, adversarial Capture the Flag framework: combining depth-aware color tracking, dynamic A* re-routing, and concurrent control threads to deliver robust performance in competitive, real-world robot encounters.

III. EXPERIMENT DESIGN

To evaluate the effectiveness of our autonomous Capture the Flag system, we devised a comprehensive, experimental framework comprising simulation-based prototyping followed by real-world implementation. This two-step process enabled us to validate our algorithms in a controlled environment before deploying them on physical hardware.

A. *Experimental Setup: Simulation Phase*

In the first phase of development, we implemented a Python-based simulation to model the capture-the-flag scenario. This environment was designed to validate our core pursuit-evasion dynamics and evaluate the viability of our interception strategy before transitioning to the physical system.

- **Grid Arena:** The simulation environment is a discrete 10×10 grid where each cell corresponds to a $0.5 \text{ m} \times 0.5 \text{ m}$ area. The attacker and defender navigate this space, and their positions are tracked using sub-cell floating-point coordinates to simulate continuous motion during updates.
- **Agents:** The attacker is user-controlled via keyboard input and can move in any of eight directions (including diagonals). The defender is autonomous, using an A* path-planning algorithm to pursue the attacker. Both agents are modeled as circular entities with a defined capture radius.
- **Path Planning:** The defender uses the A* search algorithm to calculate the shortest path to the attacker's current location or intercept path, updating its trajectory in real time. The attacker, by contrast, follows user inputs rather than algorithmic planning.
- **Flag and Drop Zone:** A single flag and drop zone are randomly placed on the grid at the beginning of each simulation, ensuring a minimum distance from the agents to allow fair gameplay and realistic pursuit dynamics.

- **Obstacles:** One static obstacle and one dynamic obstacle are placed in each simulation trial. These serve to evaluate the defender’s ability to compute collision-free paths and anticipate the attacker’s movement within a constrained environment.
- **Visualization:** Real-time feedback is provided using `matplotlib`, which displays the grid, obstacle locations, agent positions, planned paths, and status messages such as flag capture or interception events.

This simulation phase allowed iterative debugging and tuning of our interception algorithm, particularly the defender’s pursuit logic under obstacle constraints. It also provided a controlled environment for quantifying metrics such as average interception time and failure cases, which helped inform the design parameters of the real-world setup.

B. Experimental Setup: Real-World Implementation

Based on insights from simulation, we built a real-world Capture the Flag system in a controlled indoor environment. This setup closely replicates the simulated grid but introduces real-time constraints, hardware noise, and physical limitations that are critical for validating robustness.

- **Arena:** We constructed an 3×3 grid layout using white tape on the floor. Each large $1\text{m} \times 1\text{m}$ square cell was further divided into four equal subcells ($0.5\text{m} \times 0.5\text{m}$) for higher control granularity. The grid spans $4\text{m} \times 4\text{m}$ in total.
- **Flag and Drop Zone:** A red cylindrical flag was placed at the center of the grid. The drop zone was fixed at one of the far corners relative to the attacker’s start position.
- **Attacker Agent:** The attacker robot was a RoboMaster, controlled using the custom Python script `teleop_attacker.py`. The attacker relies entirely on human control and does not employ any onboard perception or planning, making its behavior reactive to the human operator’s strategy. This script allows the operator to:
 - Move forward and backwards
 - Move left or right
 - Rotate clockwise or counter-clockwise
- **Defender Robot:** : The defender used an NVIDIA Jetson Nano-based platform equipped with an Intel RealSense D435i depth camera. The RealSense provided synchronized RGB and depth data, enabling precise estimation of the attacker’s position through HSV-based

color segmentation and 3D localization. The defender utilized a dynamic A* path-planning algorithm to calculate the optimal intercept route based on the attacker's detected position and predicted path toward the flag or drop zone. This was implemented in the script `defender_physical_ctf_realsense.py`, which allowed the defender to autonomously detect, localize, and pursue the attacker without external supervision.

- **No Obstacles:** Unlike simulation trials, no static obstacles were placed in the physical environment to isolate core pursuit–evasion behavior and minimize environmental variability during evaluation.

The experimental setup, combining simulated trials with real-world deployment, offered a reliable foundation for evaluating the multi-agent coordination framework. By iteratively refining components such as perception, planning, and interaction logic within a controlled environment, the system was better equipped to handle the challenges of physical implementation with consistency and clarity.

IV. METHODOLOGY

The core functionality of our Capture the Flag system rests on three interconnected modules: (1) grid-based path planning using A* algorithm, (2) depth perception with the Intel RealSense camera, and (3) vision-based object detection via HSV color masking. Each module runs concurrently on the robot's onboard processor and communicates through a shared data structure to enable real-time pursuit–evasion behavior.

A. A* Path Planning and Re-planning

The defender employs A* search over a 3x3 grid graph. At each decision step:

- 1) **Graph Construction:** Nodes represent grid cell centers; edges connect three-neighborhood cells with unit cost.
- 2) **Heuristic:** We use the Manhattan distance, $h(n) = |x_n - x_{\text{goal}}| + |y_n - y_{\text{goal}}|$, which is admissible under 8-way movement.
- 3) **Path Extraction:** The defender uses an A path-planning algorithm* over the 3x3 grid to compute interception paths toward the attacker, whose position is estimated using HSV-based detection and depth sensing from the RealSense camera. Once the attacker's position is mapped to a grid cell, the defender:

- Calculates an optimal intercept path toward the attacker's current or predicted next cell.
- Extracts the next immediate cell along this path and determines the required movement command (forward, backward, strafe) based on grid differences
- Executes the move at a fixed speed for a predefined duration to ensure precise cell-to-cell transitions
- Re-computes the path dynamically every cycle, allowing real-time adjustments as the attacker moves.

If the attacker is lost from view, the defender switches to a search behavior, rotating until the attacker is reacquired.

B. Depth Perception with Intel RealSense

Accurate spatial awareness is provided by the Intel RealSense stereo depth camera mounted on the attacker's chassis:

- **Stereo Vision:** Stereo Vision: The camera captures synchronized RGB and depth frames at 30 Hz, supported by an IR projector for improved accuracy.
- **Depth Computation:** Depth Computation: The RealSense SDK processes these frames to produce a per-pixel depth map.
- **Attacker Localization:** Attacker Localization: When the attacker's red acrylic box is detected, its pixel centroid (u, v) and depth Z are used to compute its position relative to the defender using camera intrinsics:

$$X = \frac{(u - c_x) Z}{f_x}, \quad Y = \frac{(v - c_y) Z}{f_y},$$

where (c_x, c_y) and (f_x, f_y) are the camera's intrinsic parameters.

This real-world (X, Y) position is then mapped to the corresponding cell in the 3×3 grid, enabling the defender to compute interception paths.

C. Vision-Based Object Detection via HSV Masking

The defender identifies the attacker by detecting the red acrylic box using HSV color masking on the RealSense RGB stream:

- 1) **Color Space Conversion:** Frames are converted from BGR to HSV.
- 2) **Thresholding:** Pre-defined HSV bounds isolate the red color of the attacker's marker.

- 3) **Filtering:** Morphological operations (erosion, dilation) remove noise and refine detection.
- 4) **Target Extraction:** The largest red contour is selected, and its centroid provides the target's (u, v) position in the image frame.

This lightweight vision pipeline runs onboard the defender at 20–25 Hz, providing continuous position updates to the interception planner. Together with depth sensing, it enables the defender to track the attacker in real time and compute A-based intercept paths within the 3×3 grid*.

V. RESULTS

A. Simulation Results

The following figures shows four key snapshots from our Python-based simulation:

- (a) Beginning of the simulation
- (b) Initial positions of attacker, defender, flag and drop zone
- (c) Attacker capturing the flag
- (d) Attacker being captured and the game ending

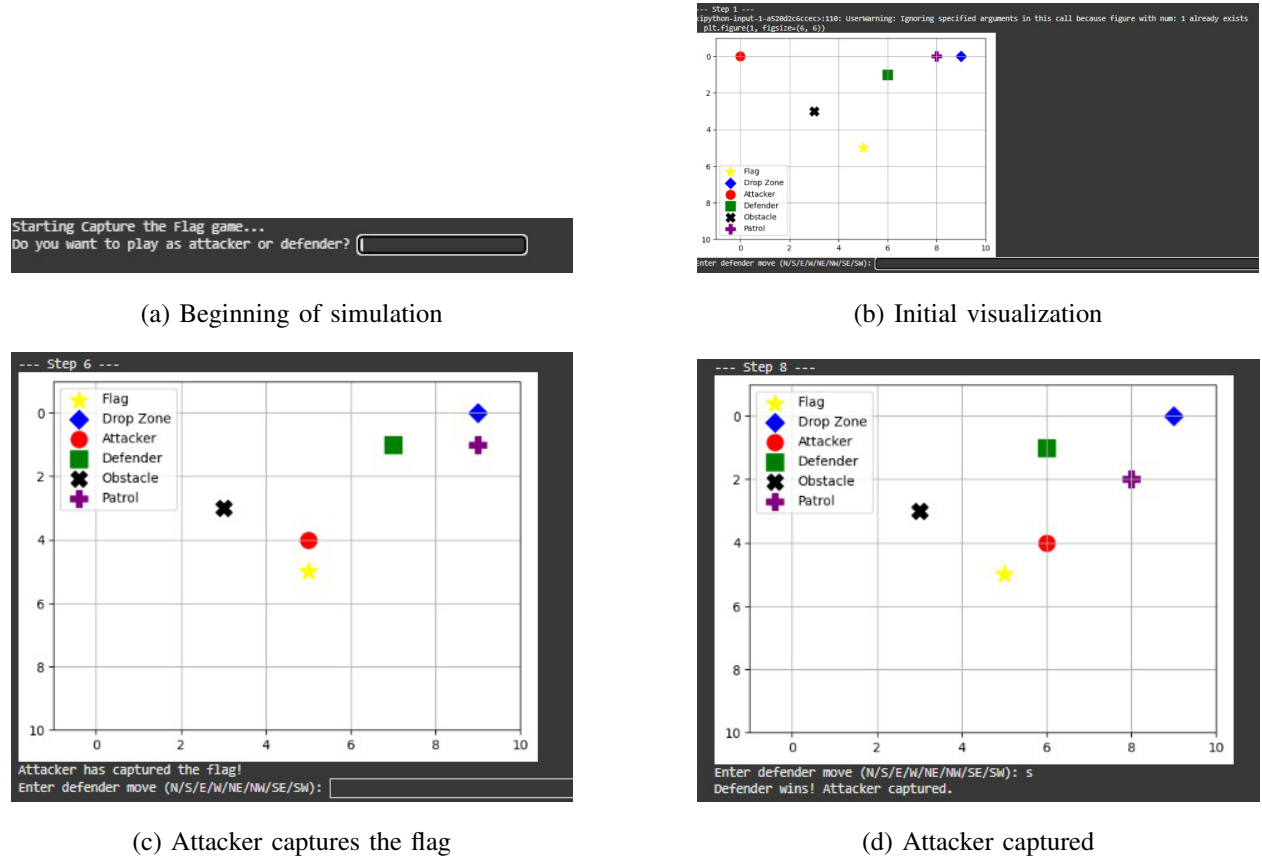


Fig. 1: Key stages in the simulation phase.

In this particular simulation run, although the attacker is capable of winning under optimal trajectories, the defender prevailed by capturing the attacker before it could reach the drop zone. As soon as the defender moved into any of the cells immediately surrounding the attacker, it was intercepted and its attempt was thwarted. This outcome underscores the defender's sole objective—to defend the flag—and demonstrates that its real-time re-planning and pursuit logic reliably secure victory whenever the attacker ventures into the critical one-cell perimeter.

B. Real world Results

In the real-world testing, the attacker robot was user controlled through keystrokes and executed a sequence where it began from its starting location, moved to the centrally located flag and the ultimately to the drop zone located in one of the grid corners. Throughout this movement, the autonomous defender robot used its onboard RealSense D455 camera to detect the red acrylic marker on the attacker robot, calculate its relative position to the attacker, and then plan interception paths using A* path planning.

The system functioned positively, with the attacker successfully completing the sequence to collect the flag and move to the drop zone with the defender actively trying to intercept its movement. It was noted that the defender robot had more success in maintaining a relative position for interception while the attacker was moving towards the drop zone versus the initial movements towards the flag. This was due to the more determined and linear nature of the attacker's movement to the drop zone, which allowed the defender to calculate effective interception paths.

Notably, some issues did persist. The RealSense D455 limited range and field of view constrained the defenders ability to detect the attacker about more than two to three cells away from its position. Additionally, the system occasionally produced false positives, particularly when the defender's camera detected the brown doors present in the background, which had similar color dominance in certain lighting conditions. This caused momentary confusion in the detection pipeline, resulting in inaccurate localization and inefficient interception paths.

VI. CONCLUSION

This project successfully demonstrated the integration of a human-operated attacker robot and an autonomous defender robot in a physical Capture the Flag game within a 3x3 grid arena. The attacker, controlled via keyboard commands, performed its tasks of flag collection and delivery

effectively, while the defender autonomously detected, tracked, and attempted to intercept the attacker using vision-based detection and grid-based path planning. The system highlighted the defender’s capacity to respond dynamically to the attacker’s position and make interception attempts, especially during the attacker’s predictable movement phases toward the drop zone.

Key contributions of this work include:

- *Teleoperated-Autonomous Integration*: Demonstrated a hybrid human-robot interaction system, with a human-operated attacker robot, and an autonomous defender robot.
- *Real-Time Visual Tracking and Path Plannin*: Implemented and validated the defender robot’s pipeline to detect in real-time using HSV color space detection and depth, and to plan A* paths for dynamic interceptions.
- *Pursuit-Evasion Dynamics in Physical Setup*: Provided insights into the performance of vision-based autonomous interception in a constrained grid, observing how the defender’s capabilities were influenced by sensor range and detection robustness.

Future Improvements

To address these limitations and extend the system’s capabilities, we propose the following developments:

a) Detection Resilience: Utilize advanced object detection methods (for example, deep learning-based models) to improve resilience to false positives and range of detection and performance in cropped or expansive visual landscapes.

b) Limited Range: To Improve the RealSense’s limited range to the blindspot, consider adding more sensors, or using additional higher fidelity sensors to assist in keeping the adversary in the sensor range of the defender.

c) Better Predictions: Include motion prediction methods for enhancement of the defender planning algorithm to enable anticipatory defence: thinking forward rather than just relying upon present detected positions.

d) Real-World Environment Generalization: Consider modifying the system to run effectively in a range of real-world environments by improving its ability to sense in different lighting conditions and including mechanisms to detect and filter background elements that would not be detected (ex. walls, doors) so false detections do not occur.

By overcoming these areas of improvement, the system can become a more positive and effective robotic pursuit-evasion system, capable of completing more complicated scenarios consisting of larger grids and multiple dynamic objects.

REFERENCES

- [1] D. P. Y. Barfeh and E. Ramos, "Color detection in autonomous robot-camera," *J. Phys.: Conf. Ser.*, vol. 1169, no. 1, p. 012048, Feb. 2019. [Online]. Available: https://www.researchgate.net/publication/331746389_Color_Detection_in_Autonomous_Robot-Camera
- [2] B. Artono, W. Nugroho, and R. Wahyudi, "Color-based image processing for autonomous human following trolley robot navigation with camera vision," *J. Comput. Sci. Eng.*, vol. 5, no. 1, pp. 20–38, Apr. 2024. [Online]. Available: https://www.researchgate.net/publication/381477406_Color-Based_Image_Processing_for_Autonomous_Human_Following_Trolley_Robot_Navigation_with_Camera_Vision
- [3] TJPRC Publication, "Cycle time optimized path planning for industrial robot using RoboMaster," 8 pp., 2019. [Online]. Available: https://www.academia.edu/35534390/CYCLE_TIME_OPTIMIZED_PATH_PLANNING_FOR_INDUSTRIAL_ROBOT_USING_ROBOMASTER?source=swp_share
- [4] A. P. R. Anilkumar *et al.*, "Design of the path following robot," *Int. J. Innov. Res. Technol.*, 8 pp., 2020. [Online]. Available: https://ijirt.org/publishedpaper/IJIRT154978_PAPER.pdf
- [5] S. Bhat, P. Kini, A. Nair, and N. Athavale, "Color tracking robot," *Int. J. Recent Res. Math. Comput. Sci. Inf. Technol.*, vol. 3, no. 1, pp. 18–22, Apr. 2016. [Online]. Available: <https://www.paperpublications.org/upload/book/Color%20Tracking%20Robot-710.pdf>